

TARGET URL:
http://demo.testfire.net/

SCAN DATE:
2025-11-29 09:23:31

REPORT GENERATED:
2025-12-07 13:08:22

Scan Summary

Risk Level	Count
High	0
Medium	3
Low	4
Informational	0
Total	7

Detailed Findings

Cross-Domain JavaScript Source File Inclusion

Low Risk

Predicted Score: 14.01

Confidence: Medium

URL: http://demo.testfire.net/index.jsp?content=personal_investments.htm

Description:

The page includes one or more script files from a third-party domain.

Solution:

Ensure JavaScript source files are loaded from only trusted sources, and the sources can't be controlled by end users of the application.

Absence of Anti-CSRF Tokens

Medium Risk

Predicted Score: 12.77

Confidence: Low

URL: http://demo.testfire.net/feedback.jsp

Description:

No Anti-CSRF tokens were found in a HTML submission form.

A cross-site request forgery is an attack that involves forcing a victim to send an HTTP request to a target destination without their knowledge or intent in order to perform an action as the victim. The underlying cause is application functionality using predictable URL/form actions in a repeatable way. The nature of the attack is that CSRF exploits the trust that a web site has for a user. By contrast, cross-site scripting (XSS) exploits the trust that a user has for a web site. Like XSS, CSRF attacks are not necessarily cross-site, but they can be. Cross-site request forgery is also known as CSRF, XSRF, one-click attack, session riding, confused deputy, and sea surf.

CSRF attacks are effective in a number of situations, including:

- * The victim has an active session on the target site.
- * The victim is authenticated via HTTP auth on the target site.
- * The victim is on the same local network as the target site.

CSRF has primarily been used to perform an action against a target site using the victim's privileges, but recent techniques have been discovered to disclose information by gaining access to the response. The risk of information disclosure is dramatically increased when the target site is vulnerable to XSS, because XSS can be used as a platform for CSRF, allowing the attack to operate within the bounds of the same-origin policy.

Solution:

Phase: Architecture and Design

Use a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid.

For example, use anti-CSRF packages such as the OWASP CSRFGuard.

Phase: Implementation

Ensure that your application is free of cross-site scripting issues, because most CSRF defenses can be bypassed using attacker-controlled script.

Phase: Architecture and Design

Generate a unique nonce for each form, place the nonce into the form, and verify the nonce upon receipt of the form. Be sure that the nonce is not predictable (CWE-330).

Note that this can be bypassed using XSS.

Identify especially dangerous operations. When the user performs a dangerous operation, send a separate confirmation request to ensure that the user intended to perform that operation.

Note that this can be bypassed using XSS.

Use the ESAPI Session Management control.

This control includes a component for CSRF.

Do not use the GET method for any request that triggers a state change.

Phase: Implementation

Check the HTTP Referer header to see if the request originated from an expected page. This could break legitimate functionality, because users or proxies may have disabled sending the Referer for privacy reasons.

Reference:

https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

<https://cwe.mitre.org/data/definitions/352.html>

Missing Anti-clickjacking Header

Medium Risk

Predicted Score: 11.0

Confidence: Medium

URL: http://demo.testfire.net/

Description:

The response does not protect against 'ClickJacking' attacks. It should include either Content-Security-Policy with 'frame-ancestors' directive or X-Frame-Options.

Solution:

Modern Web browsers support the Content-Security-Policy and X-Frame-Options HTTP headers. Ensure one of them is set on all web pages returned by your site/app.

If you expect the page to be framed only by pages on your server (e.g. it's part of a FRAMESET) then you'll want to use SAMEORIGIN, otherwise if you never expect the page to be framed, you should use DENY. Alternatively consider implementing Content Security Policy's "frame-ancestors" directive.

Reference:

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Frame-Options>

Server Leaks Version Information via "Server" HTTP Response Header Field

Low Risk

Predicted Score: 9.88

Confidence: High

URL: http://demo.testfire.net/

Description:

The web/application server is leaking version information via the "Server" HTTP response header. Access to such information may facilitate attackers identifying other vulnerabilities your web/application server is subject to.

Solution:

Ensure that your web server, application server, load balancer, etc. is configured to suppress the "Server" header or provide generic details.

Reference:

<https://httpd.apache.org/docs/current/mod/core.html#servertokens>

[https://learn.microsoft.com/en-us/previous-versions/msp-n-p/ff648552\(v=pandp.10\)](https://learn.microsoft.com/en-us/previous-versions/msp-n-p/ff648552(v=pandp.10))

<https://www.troyhunt.com/shhh-dont-let-your-response-headers/>

Content Security Policy (CSP) Header Not Set

Medium Risk

Predicted Score: 5.25

Confidence: High

URL: http://demo.testfire.net/

Description:

Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on that page — covered types are JavaScript, CSS, HTML frames, fonts, images and embeddable objects such as Java applets, ActiveX, audio and video files.

Solution:

Ensure that your web server, application server, load balancer, etc. is configured to set the Content-Security-Policy header.

Reference:

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP>

https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html

<https://www.w3.org/TR/CSP/>

<https://w3c.github.io/webappsec-csp/>

<https://web.dev/articles/csp>

<https://caniuse.com/#feat=contentsecuritypolicy>

<https://content-security-policy.com/>

X-Content-Type-Options Header Missing

Low Risk

Predicted Score: 5.25

Confidence: Medium

URL: http://demo.testfire.net/

Description:

The Anti-MIME-Sniffing header X-Content-Type-Options was not set to 'nosniff'. This allows older versions of Internet Explorer and Chrome to perform MIME-sniffing on the response body, potentially causing the response body to be interpreted and displayed as a content type other than the declared content type. Current (early 2014) and legacy versions of Firefox will use the declared content type (if one is set), rather than performing MIME-sniffing.

Solution:

Ensure that the application/web server sets the Content-Type header appropriately, and that it sets the X-Content-Type-Options header to 'nosniff' for all web pages.

If possible, ensure that the end user uses a standards-compliant and modern web browser that does not perform MIME-sniffing at all, or that can be directed by the web application/web server to not perform MIME-sniffing.

Reference:

[https://learn.microsoft.com/en-us/previous-versions/windows/internet-explorer/ie-developer/compatibility/gg622941\(v=vs.85\)](https://learn.microsoft.com/en-us/previous-versions/windows/internet-explorer/ie-developer/compatibility/gg622941(v=vs.85))

<https://owasp.org/www-community/Security-Headers>

Cookie without SameSite Attribute

Low Risk

Predicted Score: Unprofiled

Confidence: Medium

URL: http://demo.testfire.net/

Description:

A cookie has been set without the SameSite attribute, which means that the cookie can be sent as a result of a 'cross-site' request. The SameSite attribute is an effective counter measure to cross-site request forgery, cross-site script inclusion, and timing attacks.

Solution:

Ensure that the SameSite attribute is set to either 'lax' or ideally 'strict' for all cookies.

Reference:

<https://datatracker.ietf.org/doc/html/draft-ietf-httpbis-cookie-same-site>

Information Disclosure - Suspicious Comments

Predicted Score: Unprofiled

Confidence: Low

URL: http://demo.testfire.net/index.jsp?content=inside_jobs.htm

Description:

The response appears to contain suspicious comments which may help an attacker.

Solution:

Remove all comments that return information that may help an attacker and fix any underlying problems they refer to.

Modern Web Application

Predicted Score: N/A

Confidence: Medium

URL: http://demo.testfire.net/disclaimer.htm?url=http://www.microsoft.com

Description:

The application appears to be a modern web application. If you need to explore it automatically then the Ajax Spider may well be more effective than the standard one.

Solution:

This is an informational alert and so no changes are required.

Session Management Response Identified

Predicted Score: N/A

Confidence: Medium

URL: http://demo.testfire.net/

Description:

The given response has been identified as containing a session management token. The 'Other Info' field contains a set of header tokens that can be used in the Header Based Session Management Method. If the request is in a context which has a Session Management Method set to "Auto-Detect" then this rule will change the session management to use the tokens identified.

Solution:

This is an informational alert rather than a vulnerability and so there is nothing to fix.

Reference:

<https://www.zaproxy.org/docs/desktop/addons/authentication-helper/session-mgmt-id/>